

Parity Testing Java17 -> Java25

J17 J25 Behavioural Parity — Generic Guide

A one-page, context-independent companion to the full investigation (`j25-behavioural-change-investigation-report`):



). For each behavioural change (BC): what you see, how J25 handles it, and how to prove parity.

This guide is generic — no per-context detail. Which BCs apply to a given service, the files changed, and the concrete test names live in that context's own checklist, e.g. `cpp-context-<name>/doc/j25-parity-checklist.md`.

- **Scope:** the J17J25 upgrade (Java 25 / WildFly 40 / Jakarta EE 10 / Hibernate 6 / Drools 10).
- **Catalogue:** 24 BCs — **17 Confirmed**, **3 Refuted**, **2 Mixed**, **2 Inconclusive**.
- **Two risk hotspots:** the persistence layer (DeltaSpike removed every single-result finder hand-rewritten; Hibernate 56) and access control (Drools 710, CDI/Weld 4). The framework's own request-processing code is behaviourally unchanged — almost every risk is a library/server default shift or a subtly different contract in code rewritten during the migration.

How we prove parity (method in four lines)

1. **J17 is the source of truth.** Author every characterization test green on J17 *first*.
2. **Carry the identical assertion to J25.** A red test is a *finding*, never a loosened assertion.
3. Resolve each red one of two ways: **(A) fix production** to restore J17 behaviour, or **(B) consciously accept + log** the change (when the J17 behaviour was itself a bug).
4. Track coverage per context in `cpp-context-<name>/doc/j25-parity-checklist.md`.

Disposition (generic action-class, not per-context status):

- **Fix** — generally needs a production change to restore or adjust behaviour.
- **Accept** — J17 behaviour was itself a bug; keep the J25 change, pin the new contract, log it.
- **Verify** — confirm per context on a real (Docker/WF40) run; may or may not need action.
- **Feature-gated** — only applies if the context uses the named library/feature.
- **Refuted** — not a real issue for the migrated runtime (listed separately at the end).

Catalogue

Persistence (Hibernate 56, DeltaSpike JPA)

BC	Symptom	Keep parity: J17 test	J25 fix pattern	Disposition
BC-01	Single-result finder (rewritten from a DeltaSpike <code>@Query</code>) returns null on no-match instead of throwing an empty-200 response becomes 404	Assert the finder throws <code>NoResultException</code> when nothing matches	End the finder with <code>getSingleResult()</code> so it throws again; the caller's existing catch restores the empty-200	Fix (where J17 threw)
BC-02	Inverse — a finder now using <code>getSingleResult()</code> throws 500 where DeltaSpike returned null; multi-match now <code>NonUniqueResultException</code>	Audit each finder: optional/derived finders are nullnull parity	Use <code>getResultList().stream().findFirst()</code> for optional finders; constrain or guard multi-match	Verify

BC-04	NULL column hydrated into a primitive entity field <code>PropertyAccessException (500)</code> on legacy/genesis rows (Hibernate 6 — and 5.4.24 too; not a silent-0)	Assert the load throws on <code>NULL.primitive</code>	Widen plain fields <code>Long</code> with a null-safe getter. For a primitive <code>@Version</code> field keep <code>int</code> and backfill <code>NULL0</code> via a Liquibase changeset — do not widen <code>@Version</code> (breaks <code>@MapsId</code> children). Retire the J17 throw test as an accepted divergence; pin the new default-value contract	Fix / Accept
BC-05	JSQL field <code>!= null</code> compiles to <code><> null</code> UNKNOWN silently empty result set	Assert the guarded query returns the expected rows	Drop the redundant guard: <code>!= null</code> and <code>= true = true</code> (wrappers: <code>IS NOT NULL AND ...</code>)	Fix
BC-06	Stricter session boundaries lazy-collection access outside a tx <code>LazyInitializationException (500)</code>	Service-tier test hitting a lazy collection with no <code>@Transactional</code> (repo-tier tests miss it)	Add <code>@Transactional</code> to query-view service methods that touch lazy collections	Fix
BC-24	pgjdbc 42.342.7 <code>SQLState</code> / temporal edge drift	Pin <code>SQLState</code> / temporal round-trip	Confirm on a live run	Verify

Access control (Drools, CDI/Weld)

BC	Symptom	Keep parity: J17 test	J25 fix pattern	Disposition
BC-20	Drools test harness silently loads 0 rules deny-tests pass vacuously (false confidence)	Assert rule count > 0 in the harness before trusting deny assertions	Add a rule-count assertion to the shared access-control test base	Fix (harness)

(BC-03 and BC-14 also live here but were refuted — see the refuted list at the end.)

Serialization / JSON / REST

BC	Symptom	Keep parity: J17 test	J25 fix pattern	Disposition
BC-08	Jackson <code>jsr310</code> renders <code>'Z'</code> as <code>ZoneOffset.UTC</code> — <code>ZonedDateTime</code> identity drift (equals/toString, map keys)	Pin exact serialized string / equality on a <code>ZonedDateTime</code> field	Normalise the zone, or update assertions to the offset form	Fix
BC-11	CORRECTED 2026-08-26 — PARITY, not a divergence. Via the framework helper <code>JsonObjects.createObjectBuilder().add(key, (String) null)</code> throws <code>NullPointerException</code> : Value in <code>JsonObjects</code> name/value pair cannot be null on both J17/glassfish and J25/Parsson — the helper has its own null-guard, so glassfish did NOT silently accept it. Empirically verified on <code>cpp-context-notification-notify</code> (J17 and J25 produce the identical throw). A nullable value reaching <code>add()</code> (e.g. <code>e.getMessage()</code> , which can be null) is a pre-existing latent bug that NPEs the same on both runtimes — NOT a J25 regression.	Capture the J17 behaviour first (it throws) and assert J25 throws identically.	None for parity. Optional hardening: null-guard nullable values at the call sites — but apply to BOTH branches, else J25 would diverge from J17.	Refuted / parity (was: Fix)
BC-12	RESTEasy 36/7 engine swap; WARs must exclude the bundled RESTEasy	REST round-trip / packaging check	Framework-tier packaging change (exclude bundled RESTEasy)	Verify
BC-13	<code>org.json</code> + everit JSON-schema validation strictness shift — borderline payloads may newly fail	Schema-validate a borderline payload	Confirm strict-mode payloads per context	Verify

Deploy / messaging / metrics / codegen

BC	Symptom	Keep parity: J17 test	J25 fix pattern	Disposition
BC-07	Liquibase 45 rejects removed props (<code>hub.mode</code> , <code>searchPath</code>) migration job fails at deploy	— (deploy-time)	Remove the stale properties from every <code>liquibase.properties</code> (per-context sweep)	Fix (sweep)
BC-16	<code>/internal/metrics/*</code> gutted to 404	Endpoint smoke test	Known surface reduction — confirm nothing depends on it	Verify
BC-17	<code>stream_error</code> identity/dedup-hash and healthcheck message change for identical failures	Capture J17 hash/message via a deterministic failure hook, assert on J25	Partially remediated; needs the failure hook + live J17 capture	Verify (gap)
BC-18	Artemis 2.53 pins the broker URL eagerly <code>IllegalStateException</code> on <code>setBrokerURL</code> in the JMS IT harness	JMS client IT	Framework fix — drop the <code>setBrokerURL</code> call (no-arg factory defaults the URL)	Fix (framework)
BC-21	reflections 0.9.100.10.2 scanning-contract change (<code>getClassNames()</code>) — codegen may see a different type set	Codegen output test	Verify generated sources (<code>getClassNames()</code> half confirmed, scanner-count half unproven)	Verify

Feature-gated (only if the context uses that feature)

BC	Symptom	Applies to	J25 fix pattern	Disposition
BC-09	Activiti NoOpPlatformTransactionManager — atomicity/rollback lost	Activiti/Camunda contexts	Restore a real transaction manager	Feature-gated
BC-10	Activiti REST API reimplemented — null params 500, surface cut	Activiti/Camunda contexts	Re-check the REST surface per context	Feature-gated
BC-22	Apache Tika 1.283.3 MIME-detection drift	File-upload / Tika contexts	Confirm MIME detection on a live run (inconclusive)	Feature-gated
BC-15	core-domain forked from a stale base missing schema fields (identical payload, different outcome)	Contexts on the affected core-domain models	Rebase core-domain onto the current base	Fix (stale-base)
BC-19	Agent-prosecutor access silently grant deny	SJP / prosecution contexts	Rebase/cherry-pick (restored in a later milestone)	Feature-gated

Refuted — not a real issue for the migrated runtime

Kept for traceability. No code fix needed; a couple carry a cheap prophylactic guard.

BC	Claim	Why refuted	Residual guard
BC-03	Drools 710 + MVEL 2.5 recompiles the rulebase allow/deny could flip	Rules unchanged and fail-closed; independently verified	Keep *RulesTest green; pair with BC-20 rule-count assertion so a zero-rule load can't masquerade as a flip
BC-14	CDI 4 discovery-mode + legacy beans.xml could empty interceptor chains	Not reproduced on the migrated runtime	Set beans.xml to the 4.0 namespace (already part of the mechanical migration)
BC-23	Quartz 2.32.5 JobDataMap deserialization hardening	Not a live issue; scheduler-only anyway	—

BC-03 and BC-14 were the catalogue's *only two critical-rated* entries — both refuted.

Generic follow-ups (do per context)

- BC-06 — add a **service-tier** lazy-collection parity test (repo-tier tests don't catch it).
- BC-07 — sweep every context's liquibase.properties for the removed keys.
- BC-15 — rebase core-domain where a context depends on the stale-base models.
- BC-17 — add a deterministic failure hook + capture the live J17 hash/message to enable the parity IT.
- BC-20 — add a rule-count assertion to the shared Drools access-control test base.

Mechanical migration applied everywhere (not a BC): javax. jakarta.*, beans.xml 4.0, persistence.xml Jakarta 3.0, DeltaSpike repositories plain JPA/Hibernate. Full failure cookbook: the per-context upgrade playbook (cpp-context-<name>/CLAUDE.md §8).*

Evidence log — worked examples

Empirical confirmations and reusable lessons from applying this guide to real contexts. Each entry was **run**, not assumed.

Completed contexts (Platform Engineering)

Each done as two PRs — a J17 test-only PR vs team/17.104.x (branched off main; source-of-truth evidence) and a J25 PR vs team/25.104.x (same tests + any parity fix + findings docs) — each validated with a full rit (build + ITs) on both stacks, identical IT totals confirming parity. support was skipped as trivially confident.

Context	J25 PR (team/25.104.x)	J17 PR (team/17.104.x)	Jira	Notes
resulting	#122	#123	PEG-3533	Golden-master output parity; BC-04/BC-11 characterization (later retired as framework invariants)
notification	#37	#38	PEG-3534	Lean — BC-07 + BC-20

notificationnotify	#43	#44	PEG-3535	Lean — BC-07 + BC-20
system-id-mapper	#26	#27	PEG-3536	JDBC (no JPA) — BC-07 + BC-20
system-scheduling	#21	#22	PEG-3537	Quartz-backed (no JPA) — BC-07 + BC-20
system-announcement	#13	#14	PEG-3538	@Entity+persistence.xml but raw-JDBC repo — BC-07 + BC-20; dev Artemis broker.xml gap fixed
system-doc-generator	#589	#590	PEG-3401	DeltaSpikeJPA parity-clean; BC-20 COMMAND_API; QUERY_API 0-rule config quirk flagged (parity-neutral)
users-groups (reference, by Zelda)	#219	#217	PEG-3336	The canonical two-PR exemplar (full JPA context: BC-01 getSingleResult fix, BC-04 widening, BC-08/BC-17 re-tiered to framework)
hearing	#292	#293	PEG-3397	Large JPA (37 @Entity); DeltaSpikeJPA parity-clean (all getSingleResult are COUNTs; finders nullnull); BC-20 ×2
listing	#105	#106	PEG-3398	JPA; parity-clean (sole getSingleResult is a COUNT); BC-07 + BC-20 ×2; 18 changed goldens are main-advancing
mi-reportdata	#585	#587	PEG-3399	Large JPA (93 @Entity); parity-clean (findByUrn OPTIONALfindFirst; @Version long nullable=false); BC-20 ×2. J17 report ITs needed the java-17 Azurite dev-docker fix (blob seed+socat+UseDevelopmentStorage)
work-management-proxy	#28	#29	PEG-3402	Camunda, no JPA; BC-20
businessprocesses	#76	#77	PEG-3402	Camunda, small JPA; DeltaSpikeJPA parity-clean; BC-20
boxworkmanagement (wmp-combo)	—	—	PEG-3402	Nothing actionable — no rules/entities, javaxjakarta only

resulting (2026-08-25) — first context worked end-to-end

Findings written up in `cpp-context-resulting/J25-PARITY-FINDINGS.md` and `doc/j25-parity-checklist.md`; changes in PR [cpp-context-resulting#122](#). Outcome: **no functional-equivalence blocker** — the two generic hotspots turned out parity-neutral or low-exposure, and converter/event/query output is already golden-covered and byte-identical to J17 (235/260 test-resource JSON files unchanged; **0 expected/output goldens changed**), green on J25.

- **BC-04 — parity CONFIRMED on both runtimes.** Loading a NULL value into a primitive int field (`TerminalEntry.index`): **J17** (Hibernate 5.4.24) throws `PersistenceException IllegalArgumentException: Can not set int field ... to null value`; **J25** (Hibernate 6) throws `PropertyAccessException: Null value ... of primitive type`. Both 500; neither yields a silent 0 behavioural parity. The exception class/message differs (error shape, not behaviour), so BC-04 is a pre-existing latent bug, *not* a J25 regression.
- **BC-11 — divergence CONFIRMED, low exposure.** Under J25/Parsson, `JsonObjectBuilder.add(key, (String) null)` throws `NullPointerException` (J17/Glassfish silently accepted). Real behavioural change, but resulting's converters read via `getString(name)` (which throws on an absent key), so no site feeds a null not reached. Kept a regression guard test.

Reusable lessons (method refinements — apply to every context)

- **Unit tests MASK BC-04.** The unit-test schema is `hbm2ddl`-generated from the entity, so a `@Column(nullable=false)` primitive column is created NOT NULL and a plain test passes vacuously. The real exposure exists only against the nullable *Liquibase* column — the characterization test must drop the not-null first to reproduce it.
- **J17 persistence characterization tests:** load the NULL row by persisting a real entity + a JPQL query, *not* a native INSERT with a UUID id — under `HSQLDialect` the id maps to BINARY and a native UUID bind does not round-trip, so `find(id)` silently returns null and the test never exercises the hydration. (JDK 17 + Hibernate 5.4.24 required.)
- **BC-20 rule-count guard:** `StatelessKieSession` does not expose the `KieBase`, so a rule-count assertion must load via `KieServices.get().getKieClasspathContainer().getKieBase(name).getKiePackages()` and sum `getRules().size()` per kbase. Also note a partial existing guard: any `*RulesTest` with an *allow* assertion (`assertSuccessfulOutcome`) already fails on a 0-rule load. Best done once in the shared `BaseDroolsAccessControlTest.setup()` in `access-control-test-utils` to cover all contexts.
- **Golden-master is often already present — and free.** Converter tests that `assertEqualTo(readJsonFromFile("...output.json"))` against expected files that are *unchanged* from J17 constitute a passing golden master: a green J25 run proves byte-for-byte output parity at zero cost. Always verify the expected/output goldens were **not regenerated** during migration (only inputs/imports changed) — a regenerated golden silently bakes in J25 drift. Field-level with `JsonPath` pins are weaker (miss added/removed fields); tighten the risky ones to full-response `equalTo`.

Platform-Engineering lean contexts (2026-08-26) — JDBC/Quartz, not Hibernate

A cluster of Platform Engineering-owned contexts turned out **lean** from a persistence-parity standpoint: their data access is **JDBC or Quartz, not Hibernate/DeltaSpike**, so both persistence hotspots collapse and the actionable set is just **BC-07** (remove `liquibase.hub.mode`) + **BC-20** (Drools rule-count guard). Each was proven with two PRs (J17 test-only vs main + J25 tests+fix vs team/25.104.x) and a real rit on both stacks with identical IT totals.

- **system-id-mapper** — JDBC repository, no JPA. kbase `SystemId.Mapping.API`. ITs 24/0/0. [#26](#) / [#27](#) ([PEG-3536](#)).
- **system-scheduling** — Quartz-backed; the viewstore is the Quartz schema, no JPA. kbase `COMMAND_API`. `SystemSchedulingIT` 2/0/0. [#21](#) / [#22](#) ([PEG-3537](#)).

- **system-announcement — the trap:** `@Entity + persistence.xml` are present, but the repository is **raw JDBC** (`PreparedStatementWrapper + native SQL`, **no `EntityManager` anywhere**), so BC-01/02/04/05/06 are N/A by construction. Lesson: `@Entity/persistence.xml` do NOT imply a Hibernate data path — grep for `EntityManager` before assuming the persistence hotspot applies. kbase `SystemAnnouncement.API` (5 .drl). `SystemAnnouncementIT` 4/0/0. [#13](#) / [#14](#) ([PEG-3538](#)). Needed a dev-only fix too: its JMS destinations were missing from `cpp-developers-docker's Artemis broker.xml` (`artemis-healthcheck` failed with "Destination(s) ... not exist") — added on both the `java-17` and `java-25` branches so ITs run locally.
- **notification** ([#37](#) / [#38](#), [PEG-3534](#)) and **notificationnotify** ([#43](#) / [#44](#), [PEG-3535](#)) — same lean shape, BC-07 + BC-20.

system-doc-generator (2026-08-26) — DeltaSpikeJPA parity-clean; a real BC-20 catch

First Platform Engineering-owned parity context with a genuine **DeltaSpike plain-JPA** repository migration (`DocumentRepository`), and it came out **parity-clean**: `findByem.find` (nonnull — there is **no single-result `getSingleResult()` finder**, so no BC-01), `findDocumentWithCriteria` byte-identical `Criteria`, `savemerge` (client-assigned UUID same `INSERT/UPDATE`), no lazy associations (BC-06 N/A), Java-side `nonnull` predicate guards rather than JPQL (BC-05 N/A). BC-04 `int` columns are parity per the corrected BC-04 (both runtimes throw). PRs [#589](#) / [#590](#) ([PEG-3401](#)).

BC-20 gotcha — the packages attribute matches the DRL package namespace, not the resource folder. This context has two kbases: `COMMAND_API` (guarded, 5 rules) and `QUERY_API`. The `QUERY_API kmodule.xml` declares `packages="rules"` (the resource folder) but its DRL declares `package uk.gov.moj.cpp.accesscontrol;`, so `getKieBase("QUERY_API")` resolves to **0 rules**. The rule-count guard *caught* it — but it is **identical on J17 and J25**, so it is **parity-neutral**, not a J25 regression. Disposition: do **not** commit a `>0` guard for such a kbase (it is red on the J17 source of truth) and do **not** "fix" the `kmodule` on the J25 branch only (that would *create* a J17/J25 divergence). Document it and hand it to the owners as a separate, non-parity change applied to both branches. Lesson: a rule-count guard can surface a pre-existing config bug; keep the guard where the kbase genuinely has rules on J17, and treat a both-branches-0 kbase as a finding, not a parity assertion.

Also: a large `main...team/25.104.x test-resource` diff (here 14 deleted + 31 modified JSON, plus a shortened docx template list) can be **main advancing past the branch point**, not the upgrade regressing — check the merge-base and whether the changes are post-branch feature commits on main before treating them as parity findings.